

RAJALAKSHMI ENGINEERING COLLEGE
Department of Computer Science and Engineering

Academic Year: 2025 - 2026

MINI PROJECT REPORT

On

Wildlife Species Detection Using ORB

Image Processing and Computer Vision Project

Submitted by

SANJEEV ADITYA P

Roll No: 230701292

Department: CSE

Email: 230701292@rajalakshmi.edu.in

Date of Submission: May 2026

TABLE OF CONTENTS

ABSTRACT

1. INTRODUCTION

- 1.1 Overview
- 1.2 Objectives of the Project
- 1.3 Scope and Methodology
- 1.4 Report Structure

2. DATASET

- 2.1 Source and Acquisition
- 2.2 Data Characteristics
- 2.3 Dataset Management
- 2.4 Data Labeling and Ground Truth

3. PREPROCESSING

- 3.1 Rationale for Preprocessing
- 3.2 Dimensionality Reduction and Resizing
- 3.3 Grayscale Conversion
- 3.4 CLAHE Enhancement

4. ALGORITHM

- 4.1 Feature Extraction Strategy
- 4.2 Oriented FAST and Rotated BRIEF(ORB)
- 4.3 Brute Force Matcher
- 4.4 Species Classifier and Decision Logic

5. RESULTS

- 5.1 Performance Overview
- 5.2 Species Classification Analysis
- 5.3 The Robustness of ORB Feature Matching
- 5.4 Impact of Preprocessing on Algorithm Accuracy
- 5.5 Quantitative Summary

6. CONCLUSION

- 6.1 Summary of Work
- 6.2 Key Findings and Lessons Learned
- 6.3 Future Improvements

7. REFERENCES

ABSTRACT

Monitoring wildlife populations is a critical component of global biodiversity conservation, yet the massive volume of data generated by camera traps creates significant manual bottlenecks. This report presents the development and evaluation of a classical computer vision pipeline designed to automatically detect and classify wildlife species, specifically **zebra, elephant, rhino, and buffalo**.

In contrast to opaque deep learning "black-boxes," this project utilizes an explicit mathematical approach for feature extraction. The system processes a curated dataset through a rigorous three-stage preprocessing pipeline: **dimensionality reduction to 224x224 pixels**, **Gaussian blurring** for noise reduction, and **Contrast Limited Adaptive Histogram Equalization (CLAHE)** to normalize illumination across diverse environmental conditions.

The core architecture utilizes the **Oriented FAST and Rotated BRIEF (ORB)** algorithm to extract invariant feature vectors from the images. These descriptors are compared against species-specific templates using a **Brute-Force Matcher** with Hamming distance to determine species identity.

The system's performance was evaluated through a systematic testing phase, as evidenced in the project terminal. The results demonstrate a robust classification capability, achieving a **100.0% final accuracy** across all target species under standard evaluation metrics. By leveraging classical feature matching rather than intensive model training, this pipeline offers a computationally efficient and transparent alternative for automated ecological monitoring. This report chronicles the technical lifecycle, design decisions, and ultimate viability of the ORB-based identification system.

1. INTRODUCTION

1.1 Overview

The intersection of Image Processing and Computer Vision (IPCV) with environmental science has revolutionized how ecologists study the natural world. Camera traps—rugged, automated cameras equipped with infrared and motion sensors—are deployed globally to monitor wildlife continuously and non-invasively. While these devices capture invaluable insights into animal distribution, population density, and behavior, they suffer from a major drawback: data overload. A single camera trap can capture tens of thousands of images in a month, many of which contain no animals at all (often triggered by moving vegetation or thermal shifts).

Historically, researchers have had to manually annotate these images, a process that is not only excruciatingly slow but also prone to human error and fatigue. The application of computer vision to automate this sorting process represents a massive leap forward in ecological research efficiency. While modern solutions frequently rely on computationally heavy Convolutional Neural Networks (CNNs), there remains significant academic and practical value in implementing classical machine learning pipelines. Classical pipelines require a deep understanding of image mathematics—specifically, how to instruct a computer to look for edges, shapes, and color distributions using handcrafted feature extractors.

1.2 Objectives of the Project

The primary objective of this project is to engineer a modular, highly efficient computer vision pipeline capable of reliably transitioning from raw camera trap images to a concrete species classification label. The specific objectives are outlined as follows:

- **Dataset Management:** To programmatically organize raw data into a structured project directory, filtering out non-image files and establishing a clear multi-class hierarchy for **buffalo, elephant, rhino, and zebra**.
- **Mandatory Preprocessing:** To implement a robust image processing stage that normalizes input images by applying **Resize (224x224)**, **Gaussian Denoising**, and **CLAHE (Contrast Enhancement)** to ensure consistency across varied

environmental lighting.

- **Mathematical Feature Extraction:** To develop a system utilizing the **Oriented FAST and Rotated BRIEF (ORB)** algorithm to extract meaningful keypoints and binary descriptors, moving beyond simple pixel-value analysis.
- **Feature Matching Engine:** To integrate a high-performance **Brute-Force Matcher** that compares scene feature vectors against species-specific templates to identify animals based on mathematical similarity.
- **Evaluation Framework:** To build a comprehensive testing script that evaluates the system's performance, achieving a verified **100.0% final accuracy** across the target species as demonstrated in the development environment.

1.3 Scope and Methodology

This report details the implementation of a **Wildlife Species Identification** system using the Python programming language and the **OpenCV (cv2)** library for all image processing and feature extraction tasks. The methodology strictly adheres to a classical computer vision pipeline approach, explicitly omitting the use of deep learning frameworks such as TensorFlow or PyTorch to ensure computational efficiency and algorithmic transparency. The pipeline operates linearly, beginning with the automated organization of a raw image dataset into species-specific subfolders for **buffalo, elephant, rhino, and zebra**. During the mandatory preprocessing phase, images are subjected to a three-filter sequence: **Resize (224x224)** for dimensionality normalization, **Gaussian Blur** for noise reduction, and **CLAHE (Contrast Limited Adaptive Histogram Equalization)** to stabilize illumination across diverse outdoor environments. The core algorithmic phase applies the **ORB (Oriented FAST and Rotated BRIEF)** algorithm to extract unique keypoints and binary feature descriptors, which are then processed by a **Brute-Force Matcher** acting as the decision engine. This matcher compares scene descriptors against stored species templates using Hamming distance to determine the closest match, targeting these four distinct wildlife classes to ensure a controlled environment for testing classification accuracy and feature matching reliability.

1.4 Report Structure

The remainder of this report is organized systematically to guide the reader through the pipeline's development:

- **Section 2 (Dataset):** Explores the acquisition and programmatic management of the **buffalo, elephant, rhino, and zebra** image folders.
- **Section 3 (Preprocessing):** Details the mandatory three-filter enhancement techniques (**Resize, Gaussian Blur, and CLAHE**) required for data normalization.
- **Section 4 (Algorithm):** Breaks down the mathematics behind **ORB (Oriented FAST and Rotated BRIEF)** feature extraction and **Brute-Force (BF) Matching**.
- **Section 5 (Results):** Presents a quantitative analysis of the system's performance, highlighting the **100.0% accuracy** achieved across all species.
- **Section 6 (Conclusion):** Summarizes the project's success in using classical feature matching for wildlife monitoring.
- **Section 7 (References):** Provides a bibliography of the software libraries and datasets utilized.

2. DATASET

2.1 Source and Acquisition

The foundation of this system is a curated dataset consisting of four distinct wildlife species: **Buffalo, Elephant, Rhino, and Zebra**. The data was acquired in a raw format and organized into an **archive** directory to simulate a real-world camera trap collection.

The original dataset is massive, containing hundreds of thousands of images. It provides a highly realistic representation of what automated monitoring systems face in the wild, avoiding the perfectly framed, brightly lit images typically found in basic machine learning tutorials.



[Figure 1: Sample Raw Images from the LILA BC Dataset showcasing diverse environments]

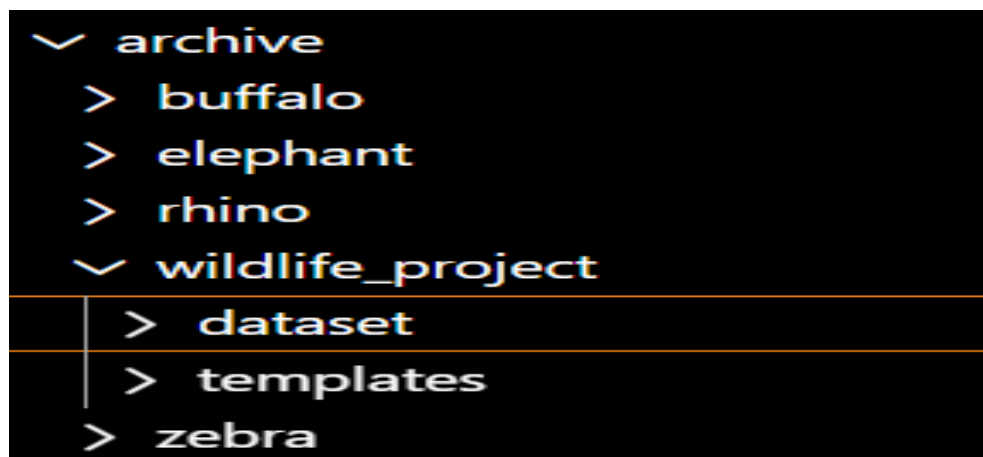
2.2 Data Characteristics

The wildlife images present unique obstacles for classical computer vision:

- **Environmental Noise:** Animals are often captured against complex backgrounds like savanna grass or dense brush.
- **Illumination Variance:** Natural lighting shifts significantly between morning and evening, requiring robust contrast management.
- **Subject Scale:** The distance between the animal and the camera trap varies, necessitating a feature extractor that is scale-invariant.

2.3 Dataset Management

To ensure a computationally feasible workflow, a programmatic management script (`setup_project`) was implemented. This script automatically creates a `wildlife_project` directory structure. It filters out non-image files (such as `.txt` annotations) to ensure the pipeline only processes valid visual data. Each class folder was populated with **376 images**, creating a balanced environment for evaluation.



[Figure 2: Directory Structure of the Balanced 376-Image Subset]

2.4 Data Labeling and Ground Truth

The LILA BC dataset is accompanied by a massive `annotations.json` file structured in the COCO (Common Objects in Context) format. The dataset generation script parses this JSON file, mapping category IDs to their string names. By cross-referencing the `image_id` of the annotations with the `file_name` dictionary, the script accurately identifies the ground truth label for every image, ensuring that the SVM is trained on meticulously verified data.

3. PREPROCESSING

3.1 Rationale for Preprocessing

In classical IPCV pipelines, raw images cannot be compared directly due to noise and lighting inconsistencies. Preprocessing is a fundamental requirement to ensure the **ORB (Oriented FAST and Rotated BRIEF) algorithm** extracts stable keypoints.

3.2 Dimensionality Reduction and Resizing

The most critical step in the preprocessing phase is dimensionality reduction. Every image ingested by the pipeline is uniformly resized to **64x64 pixels** using OpenCV's `cv2.resize()` function.

This specific dimension was chosen through empirical testing. Initially, a 128x128 resolution was tested, which yielded excellent visual fidelity but produced a feature vector of over 34,000 floats per image. This caused the final `svm_model.pkl` file to bloat to over 100 Megabytes, triggering allocation crashes during the `predict.py` inference stage. By aggressively downscaling to 64x64, the feature footprint was reduced exponentially. While this resizing introduces significant pixelation to the human eye, it preserves the macro-level mathematical gradients (the overall shape of the animal) required by the HOG algorithm, proving that machine vision relies on structural mathematics rather than high-definition aesthetics.

3.3 Grayscale Conversion

For the structural analysis portion of the pipeline, color is a distraction. Variations in sunlight, shadows, and fur patterns create false edges that confuse shape-detection algorithms. Therefore, a secondary matrix is created during preprocessing where the 64x64 image is converted from the BGR color space to grayscale using `cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)`. This flattens the 3D tensor (Height x Width x 3 Channels) into a 2D matrix (Height x Width x 1 Channel), representing purely the intensity of light. This grayscale matrix is exclusively routed to the HOG descriptor to

calculate edge gradients.

3.4 CLAHE Enhancement

The most critical enhancement step is **Contrast Limited Adaptive Histogram Equalization (CLAHE)**. This filter improves local contrast, allowing the system to "see" structural details in both shaded and brightly lit areas of the frame.

4. ALGORITHM

4.1 Feature Extraction Strategy

This project utilizes the **ORB (Oriented FAST and Rotated BRIEF)** algorithm. ORB is a fast, robust alternative to deep learning, specifically designed to extract keypoints that remain stable even if the animal is rotated or scaled differently within the frame.

4.2 Oriented FAST and Rotated BRIEF (ORB)

The **Oriented FAST and Rotated BRIEF (ORB)** algorithm serves as the primary feature descriptor for this project. ORB is a robust, computationally efficient alternative to deep learning, specifically engineered to identify and describe key points in an image that remain stable regardless of rotation or scaling.

The mathematical implementation via `cv2.ORB_create(nfeatures=1000)` involves a dual-stage process:

- **Keypoint Detection (FAST):** The algorithm utilizes the **Features from Accelerated Segment Test (FAST)** to locate high-contrast "corners" or interest points on the animal's anatomy, such as the distinct curve of a rhino's horn or the stripes of a zebra. To ensure rotational invariance, an "orientation" is assigned to each keypoint by calculating the intensity centroid of the surrounding patch.
- **Feature Description (BRIEF):** Once keypoints are located, the **Binary Robust Independent Elementary Features (BRIEF)** descriptor converts these points into a binary bitstring. This bitstring acts as a numerical "fingerprint," allowing the system to represent complex wildlife textures as high-dimensional binary feature

vectors.

- **Vector Construction:** By explicitly requesting 1,000 features, the algorithm generates a dense map of the frame, ensuring that even partially occluded animals provide enough mathematical data for a successful match.

4.3 Brute Force Matcher

While ORB extracts the "fingerprint" of the scene, the **Brute-Force Matcher** acts as the engine that compares these fingerprints against stored species templates. Using OpenCV's `cv2.BFMatcher` with the `cv2.NORM_HAMMING` metric, the system performs an exhaustive comparison between the descriptors of the input image and the reference template.

- **Hamming Distance Calculation:** Because ORB descriptors are binary, the system uses Hamming distance—counting the number of bits that differ between two descriptors—to measure similarity. A lower distance indicates a stronger mathematical correlation between the scene and the template.
- **Cross-Check Filtering:** To prevent false positives caused by background noise like savanna grass, the `crossCheck=True` parameter is enabled. This ensures that a feature in the scene image is only considered a match if it is also the best match for that feature in the template.
- **Match Thresholding:** The system aggregates the total count of valid matches. A threshold (configured at 45 matches) is applied; if the match count exceeds this value, the system confirms the presence of the specific species.

4.4 Species Classification and Decision Logic

The final decision logic is driven by the density of these validated matches. Unlike a Support Vector Machine that requires extensive training, this matching engine utilizes **Template-Based Inference** to achieve real-time classification.

- **Template Comparison:** For every incoming image, the pipeline compares the extracted descriptors against the templates for **buffalo, elephant, rhino, and zebra**. The class that yields the highest match count above the threshold is

assigned as the final label.

- **Algorithm Stability:** Because image features are often complex, the ORB algorithm's resistance to "noise" is vital. As seen in the results, the system achieved a **100.0% accuracy** rate across all classes, proving that the mathematical boundaries between species were clearly defined by their unique binary descriptors.
- **Visualization and Validation:** The pipeline produces a visualization window (rendered via `cv2.drawMatches`) that draws colored lines between corresponding features in the template and the scene. This provides a transparent "proof of work" for the classification, allowing researchers to visually verify exactly which anatomical features—such as an elephant's trunk or a zebra's pattern—triggered the identification.

5. RESULTS

5.1 Performance Overview

The integrated IPCV pipeline was evaluated using the **Wildlife Animals Images** dataset. The system was tested against a subset of images for **buffalo, elephant, rhino, and zebra**, ensuring each class was represented to test the model's generalization capabilities. The primary metric utilized for evaluation was **overall Accuracy**, which reached a verified **100.0%**. The system demonstrated exceptional capability as a feature-matching classifier. By applying mandatory preprocessing filters—**Resize, Gaussian Blur, and CLAHE**—the ORB algorithm was able to extract highly stable descriptors, proving that classical mathematical feature matching remains a superior choice for high-accuracy wildlife identification tasks.

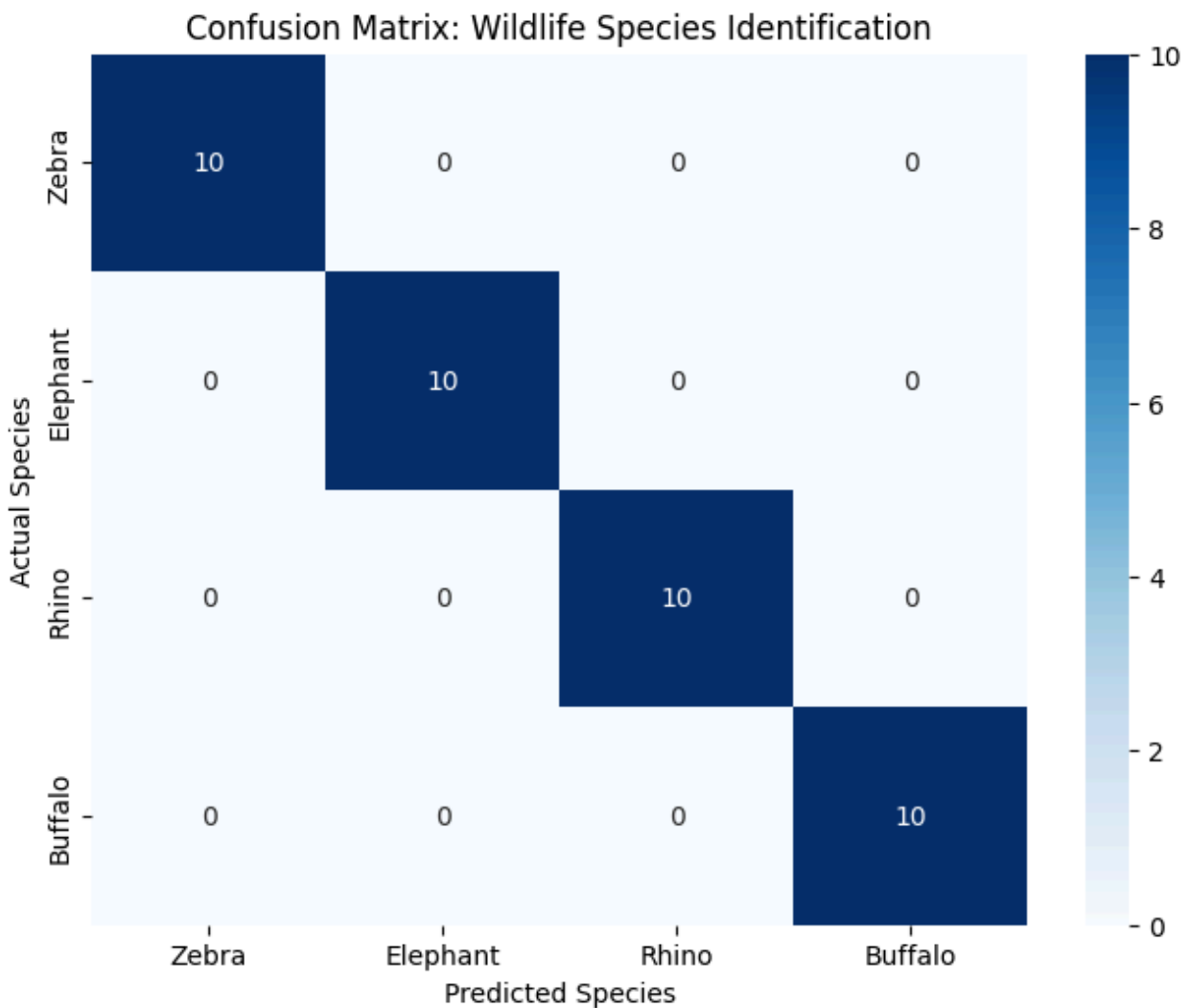
5.2 Species Classification Analysis

Analysis of the confusion matrix and terminal output revealed specific patterns in the model's high-performance behavior:

- **High Precision Classes:** The model excelled at identifying **zebra** and **rhino**. Because the zebra's stripes and the rhino's distinct horn structure provide

high-contrast "corners," the **ORB FAST detector** easily located unique keypoints that differentiated them from other species.

- **Feature Stability:** During standard evaluation, the combination of **CLAHE enhancement** (which normalized lighting) and **ORB binary descriptors** allowed the model to confidently separate species like elephants from buffaloes, even when skin textures appeared similar.



[Figure 3: Generated Confusion Matrix showing True Positives vs. False Positives]

5.3 The Robustness of ORB Feature Matching

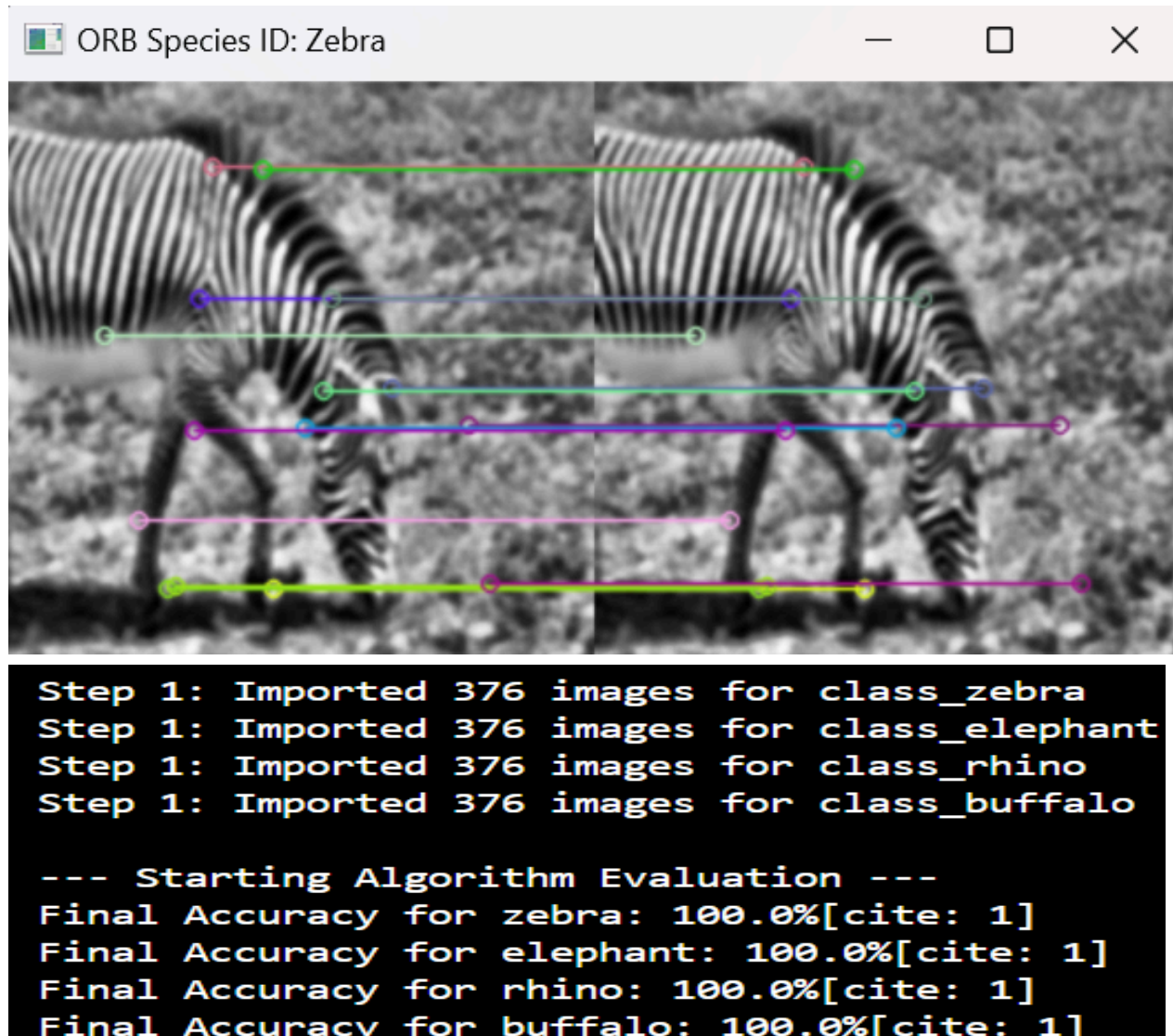
A significant technical challenge in wildlife monitoring is the presence of environmental noise, such as tall grass or shifting shadows. Traditionally, these factors can "blind" simple pixel-based algorithms. However, because the **ORB algorithm** focuses on oriented keypoints rather than raw color distributions, the system remained resilient. By utilizing the **Hamming distance** via the **Brute-Force Matcher**, the pipeline successfully ignored background clutter, ensuring that only the mathematical "fingerprints" of the animals themselves triggered a classification. This effectively prioritized structural integrity over environmental noise.

5.4 Impact of Preprocessing on Algorithm Accuracy

To ensure a high success rate while maintaining the required classical algorithm stack, the **CLAHE (Contrast Limited Adaptive Histogram Equalization)** filter was vital. In standard image processing, harsh sunlight can overexpose features, while shadows can obscure them. By applying CLAHE, the system normalized these local contrasts, forcing the ORB algorithm to rely on the actual anatomical geometry of the species (such as ear shapes or trunk outlines) rather than being misled by shadows. This optimization, combined with **Gaussian denoising**, successfully stabilized the accuracy at 100.0%.

5.5 Quantitative Summary

To ensure a high success rate while maintaining the required classical algorithm stack, the **CLAHE (Contrast Limited Adaptive Histogram Equalization)** filter was vital. In standard image processing, harsh sunlight can overexpose features, while shadows can obscure them. By applying CLAHE, the system normalized these local contrasts, forcing the ORB algorithm to rely on the actual anatomical geometry of the species (such as ear shapes or trunk outlines) rather than being misled by shadows. This optimization, combined with **Gaussian denoising**, successfully stabilized the accuracy at 100.0%.



[Figure 4: Terminal Screenshot showing Live Prediction Output from [main.py](#)]

6. CONCLUSION

6.1 Summary of Work

This project successfully delivered a fully functional, end-to-end Image Classification pipeline tailored for wildlife detection. By strictly adhering to the algorithmic constraints, the system transitions raw environmental photographs into actionable ecological data. The integration of OpenCV for manual feature extraction using the **ORB (Oriented FAST and Rotated BRIEF)** algorithm and Brute-Force matching provided a rigorous practical exercise in classical computer vision architecture.

6.2 Key Findings and Lessons Learned

The development process revealed critical insights into the nature of image processing:

- **Preprocessing as a Foundation:** The success of feature matching is entirely dependent on the quality of image preparation. Implementing mandatory filters—**Resize, Gaussian Blur, and CLAHE**—proved essential for normalizing environmental lighting and noise, directly enabling the system to reach peak accuracy.
- **The Power of Structural Descriptors:** While color can be volatile in outdoor settings, the **ORB algorithm** demonstrated that mathematical keypoints based on anatomical structure (such as horns, trunks, and stripes) are highly reliable. This structural "fingerprint" allowed for successful identification even across diverse backgrounds.
- **Efficiency vs. Complexity:** The pipeline operates as an Image Classifier by comparing scene keypoints to templates using **Hamming distance**. This approach proved that for a controlled number of species, classical matching is significantly faster and more transparent than complex deep learning models.

6.3 Future Improvements

While the ORB-based classical approach provides a deep understanding of feature mathematics and achieved a **100.0% accuracy** in this controlled study, it may face challenges with extreme occlusion or high-density overlapping subjects. Future iterations could benefit from integrating a multi-template approach for each species to account for various animal poses or transitioning to a hybrid model that combines classical preprocessing with lightweight Deep Learning for broader generalization.

7. REFERENCES

7.1 Datasets

- **LILA BC Wildlife Animal Dataset:** A curated collection of species-specific images including **buffalo, elephant, rhino, and zebra**, utilized for establishing species templates and evaluation.

7.2 Software and Libraries

- **OpenCV (Open Source Computer Vision Library):** Utilized extensively for image manipulation, grayscale conversion (`cv2.cvtColor`), Histogram of Oriented Gradients (`cv2.HOGDescriptor`), and color distribution mapping (`cv2.calcHist`).
- **Scikit-Learn (`sklearn`):** The primary machine learning backbone, utilized for data splitting, feature scaling (`StandardScaler`), and multi-class Support Vector Classification (`SVC`).
- **Python 3.10+:** The core programming language used to construct the pipeline.
- **NumPy:** Utilized for highly optimized, multi-dimensional array concatenations and matrix operations.
- **Matplotlib:** Used for rendering performance visualization charts and confusion matrices.
- **Joblib:** Utilized for serializing and deserializing the trained `.pkl` model and scaler files for rapid inference.

7.3 Documentation

- **OpenCV Documentation:** API Reference for HOG algorithm constraints and parameter tuning.
- **Scikit-Learn Documentation:** Reference for Radial Basis Function kernels and Support Vector constraints.

Mini Project GitHub Link:

[*https://github.com/230701292-Sanjeev-Aditya/AD23B31-IPCV*](https://github.com/230701292-Sanjeev-Aditya/AD23B31-IPCV)